

# MyBatis

鸣谢：波波酱老师【1小时入门MyBatis框架-CRUD极简案例教学-特种兵后端系列】

[https://www.bilibili.com/video/BV1HH4y1o7Wr?vd\\_source=b7f14ba5e783353d06a99352d23ebca9](https://www.bilibili.com/video/BV1HH4y1o7Wr?vd_source=b7f14ba5e783353d06a99352d23ebca9)



## 一、简介

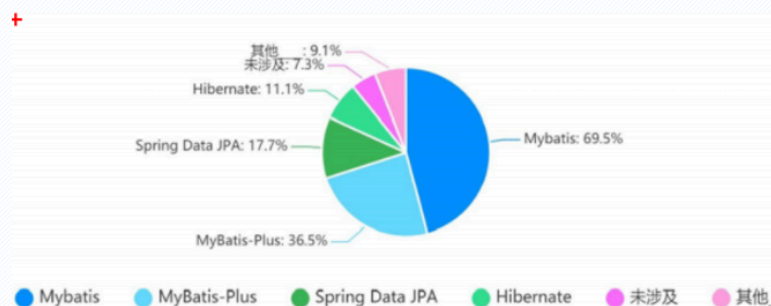
概念：MyBatis 是一款优秀的==持久层框架[DAO]==，用于简化 JDBC 开发

1. MyBatis **怎么念**?

2. MyBatis 本是 Apache 的一个开源项目iBatis，2013年11月迁移到Github。

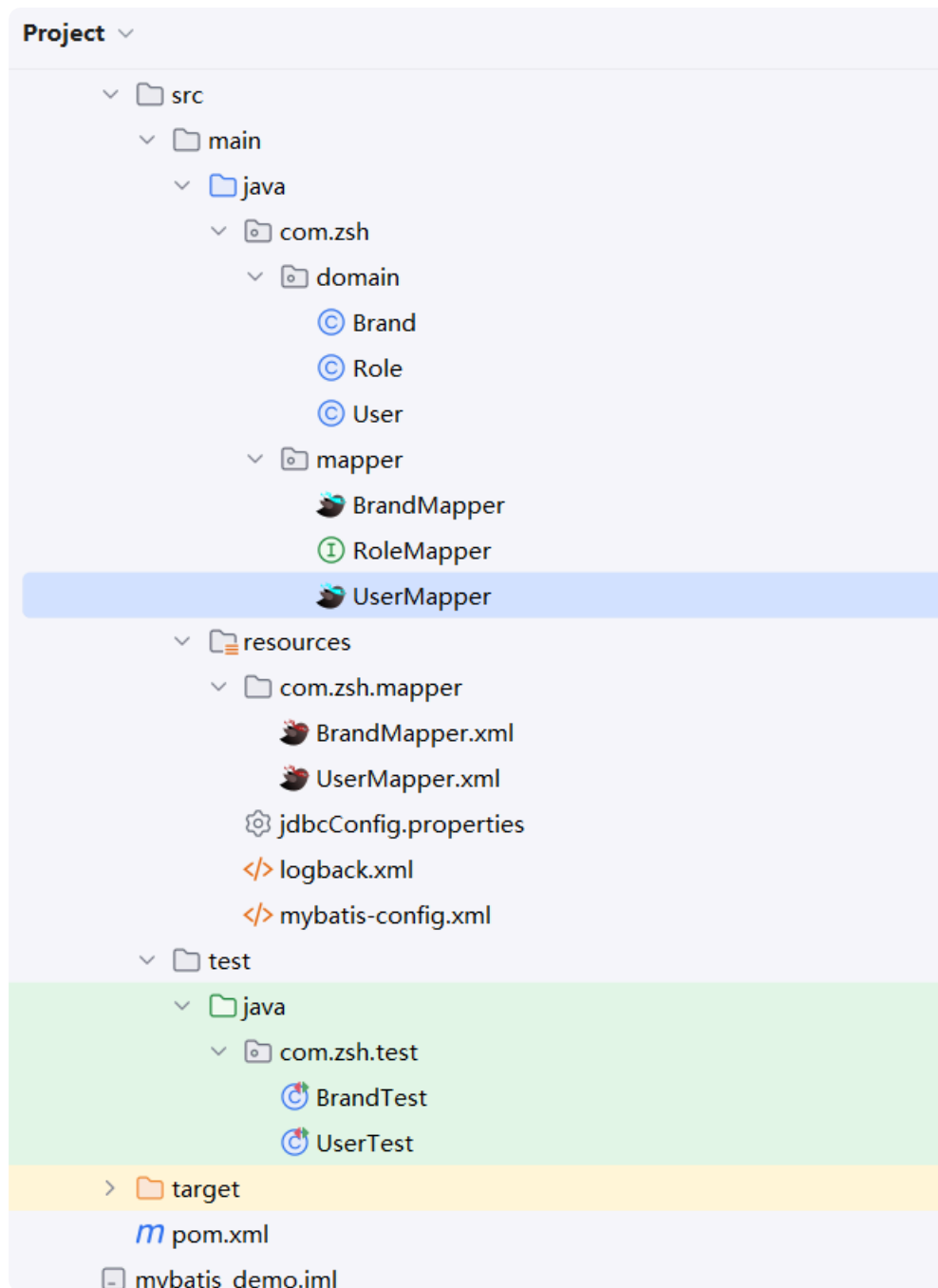
3. 官网： <https://mybatis.org/mybatis-3>

MyBatis 是目前**JAVA最主流**持久层框架



## 二、MyBatis入门案例：查询所有用户信息- `selectAll`

### 0.项目整体结构



### 1.导入 `mybatis.sql` 文件，初始化 `mybatis` 数据库

```
1 CREATE DATABASE IF NOT EXISTS `mybatis` DEFAULT CHARACTER SET utf8
  COLLATE utf8_bin
2
3 USE `mybatis`;
4
5 /*Table structure for table `account` */
```

```

6 DROP TABLE IF EXISTS `account`;
7 CREATE TABLE `account` (
8     `id` int(11) NOT NULL COMMENT '编号',
9     `uid` int(11) DEFAULT NULL COMMENT '用户编号',
10    `money` double DEFAULT NULL COMMENT '金额',
11    PRIMARY KEY (`id`),
12    KEY `FK_Reference_8` (`uid`),
13    CONSTRAINT `FK_Reference_8` FOREIGN KEY (`uid`) REFERENCES `user`
    (`id`)
14 ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
15
16 /*Data for the table `account` */
17 insert into `account`(`id`,`uid`,`money`) values (1,46,1000),
    (2,45,1000),(3,46,2000);
18
19 /*Table structure for table `brand` */
20 DROP TABLE IF EXISTS `brand`;
21 CREATE TABLE `brand` (
22     `id` int(11) NOT NULL AUTO_INCREMENT,
23     `brand_name` varchar(20) COLLATE utf8_bin DEFAULT NULL,
24     `company_name` varchar(20) COLLATE utf8_bin DEFAULT NULL,
25     `ordered` int(11) DEFAULT NULL,
26     `description` varchar(100) COLLATE utf8_bin DEFAULT NULL,
27     `status` int(11) DEFAULT NULL,
28     PRIMARY KEY (`id`)
29 ) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8 COLLATE=utf8_bin;
30
31 /*Data for the table `brand` */
32 insert into
    `brand`(`id`,`brand_name`,`company_name`,`ordered`,`description`,`status`) values
33 (1,'格力','格力科技有限公司',1,'格力改变世界',0),
34 (2,'华为','华为技术有限公司',2,'华为科技世界之巅',1),
35 (3,'美的','美的科技有限公司',3,'美的改变世界',1);
36
37 /*Table structure for table `role` */
38 DROP TABLE IF EXISTS `role`;
39 CREATE TABLE `role` (
40     `id` int(11) NOT NULL COMMENT '编号',
41     `rolename` varchar(30) DEFAULT NULL COMMENT '角色名称',
42     `roledesc` varchar(60) DEFAULT NULL COMMENT '角色描述',
43     PRIMARY KEY (`id`)
44 ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
45
46 /*Data for the table `role` */

```

```

47 insert into `role`(`id`,`rolename`,`roledesc`) values
48 (1,'仓库管理员','管理整个公司仓库模块'),
49 (2,'财务管理员','管理整个公司财务模块');
50
51 /*Table structure for table `user` */
52 DROP TABLE IF EXISTS `user`;
53 CREATE TABLE `user` (
54   `id` int(11) NOT NULL AUTO_INCREMENT,
55   `username` varchar(20) NOT NULL COMMENT '用户名称',
56   `password` varchar(20) DEFAULT NULL COMMENT '生日',
57   `gender` char(1) DEFAULT NULL COMMENT '性别',
58   `address` varchar(100) DEFAULT NULL COMMENT '地址',
59   PRIMARY KEY (`id`)
60 ) ENGINE=InnoDB AUTO_INCREMENT=52 DEFAULT CHARSET=utf8;
61
62 /*Data for the table `user` */
63 insert into `user`(`id`,`username`,`password`,`gender`,`address`)
64 values
65 (41,'李四','123456','男','北京'),
66 (42,'小王','1234567','女','浙江金华'),
67 (43,'大王','1234568','女','浙江杭州'),
68 (45,'王五','1234569','男','山东廊坊'),
69 (46,'老赵','1234560','男','北京朝阳区'),
70 (48,'小李','123456','女','福建福州'),
71 (49,'小强','123456','男','浙江东阳');
72
73 /*Table structure for table `user_role` */
74 DROP TABLE IF EXISTS `user_role`;
75 CREATE TABLE `user_role` (
76   `uid` int(11) NOT NULL COMMENT '用户编号',
77   `rid` int(11) NOT NULL COMMENT '角色编号',
78   PRIMARY KEY (`uid`,`rid`),
79   KEY `FK_Reference_10` (`rid`),
80   CONSTRAINT `FK_Reference_10` FOREIGN KEY (`rid`) REFERENCES `role`
81   (`id`),
82   CONSTRAINT `FK_Reference_9` FOREIGN KEY (`uid`) REFERENCES `user`
83   (`id`)
84 ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
85
86 /*Data for the table `user_role` */
87 insert into `user_role`(`uid`,`rid`) values (41,1),(45,1),(41,2);

```

2.创建空工程 `MyBatisProject`, 创建模块 `mybatis`, 在 `pom.xml` 中导入各种需要的依赖坐标, 在项目的 `resources` 目录下复制 `logback.xml` 这一配置文件

- pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>com.zsh</groupId>
8     <artifactId>mybatis</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>8</maven.compiler.source>
13         <maven.compiler.target>8</maven.compiler.target>
14         <project.build.sourceEncoding>UTF-
15 8</project.build.sourceEncoding>
16     </properties>
17
18     <dependencies>
19         <!-- mybatis -->
20         <dependency>
21             <groupId>org.mybatis</groupId>
22             <artifactId>mybatis</artifactId>
23             <version>3.5.5</version>
24         </dependency>
25         <!-- mysql 驱动 -->
26         <dependency>
27             <groupId>mysql</groupId>
28             <artifactId>mysql-connector-java</artifactId>
29             <version>8.0.30</version>
30         </dependency>
31         <!-- junit 驱动 -->
32         <dependency>
33             <groupId>junit</groupId>
34             <artifactId>junit</artifactId>
35             <version>4.13</version>
36             <scope>test</scope>
37         </dependency>
38         <!-- 添加slf4j日志api -->
39         <dependency>
40             <groupId>org.slf4j</groupId>
41             <artifactId>slf4j-api</artifactId>
```

```

41         <version>1.7.20</version>
42     </dependency>
43     <!-- 添加logback-classic依赖 -->
44     <dependency>
45         <groupId>ch.qos.logback</groupId>
46         <artifactId>logback-classic</artifactId>
47         <version>1.2.3</version>
48     </dependency>
49     <!-- 添加logback-core依赖 -->
50     <dependency>
51         <groupId>ch.qos.logback</groupId>
52         <artifactId>logback-core</artifactId>
53         <version>1.2.3</version>
54     </dependency>
55 </dependencies>
56 </project>

```

- logback.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <configuration>
3      <!-- CONSOLE : 表示当前的日志信息是可以输出到控制台的。 -->
4      <appender name="Console"
5      class="ch.qos.logback.core.ConsoleAppender">
6          <encoder>
7              <pattern>
8                  [%level] %blue(%d{HH:mm:ss.SSS}) %cyan([%thread])
9                  %boldGreen(%logger{15}) - %msg %n
10             </pattern>
11         </encoder>
12     </appender>
13
14     <logger name="com.itbbj" level="DEBUG" additivity="false">
15         <appender-ref ref="Console"/>
16     </logger>
17
18     <!--
19         level:用来设置打印级别，大小写无关：TRACE，DEBUG，INFO，WARN，ERROR，
20         ALL 和 OFF，默认debug
21         <root>可以包含零个或多个<appender-ref>元素，标识这个输出位置将会被本日志
22         级别控制。
23     -->

```

```
20     <root level="DEBUG">
21         <appender-ref ref="Console"/>
22     </root>
23 </configuration>
```

3.创建实体层 `com.zsh.domain` , 创建实体类 `User` , 参考数据库字段编写代码

```
1 package com.zsh.domain;
2
3 public class User {
4     private Integer id;
5     private String username;
6     private String password;
7     private String gender;
8     private String address;
9
10    ...
11 }
```

4.创建数据库映射层 `com.zsh.mapper` ,创建接口 `UserMapper` ,编写 `selectAll` 方法

```
1 package com.zsh.mapper;
2
3 import com.zsh.domain.User;
4 import java.util.List;
5
6 public interface UserMapper {
7     List<User> selectAll();
8 }
```

5.在模块的 `resources-com.zsh.mapper` 目录下创建映射配置文件 `UserMapper.xml`

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.UserMapper">
6     <select id="selectAll" resultType="User">
7         select * from user
8     </select>
9 </mapper>

```

6.在模块的 `resources` 目录下创建 `mybatis` 的配置文件 `mybatis-config.xml` , 从而建立与数据库之间的连接 (**注册驱动**)

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE configuration
3     PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-config.dtd">
5 <configuration>
6     <environments default="development">
7         <environment id="development">
8             <transactionManager type="JDBC"/>
9             <dataSource type="POOLED">
10                 <property name="driver"
value="com.mysql.cj.jdbc.Driver"/>
11                 <property name="url"
value="jdbc:mysql://localhost:3306/mybatis?characterEncoding=utf8"/>
12                 <property name="username" value="root"/>
13                 <property name="password" value="123456"/>
14             </dataSource>
15         </environment>
16     </environments>
17     <mappers>
18         <mapper resource="com/zsh/mapper/UserMapper.xml"/>
19     </mappers>
20 </configuration>

```

7.在模块的 `test-java` 目录下创建 `com.zsh.test` 层, 创建 `UserTest` 类, 编码

```
1 package com.zsh.test;
2
3 import com.zsh.domain.User;
4 import com.zsh.mapper.UserMapper;
5 import org.apache.ibatis.io.Resources;
6 import org.apache.ibatis.session.SqlSession;
7 import org.apache.ibatis.session.SqlSessionFactory;
8 import org.apache.ibatis.session.SqlSessionFactoryBuilder;
9 import org.junit.Test;
10 import java.io.IOException;
11 import java.io.InputStream;
12 import java.util.List;
13
14 public class UserTest {
15     @Test
16     public void testSelectAll() throws IOException {
17         //1. 读取配置文件--在resources文件夹下可以直接读到
18         InputStream inputStream =
19 Resources.getResourceAsStream("mybatis-config.xml");
20         //2. 创建SqlSessionFactory工厂
21         SqlSessionFactory sqlSessionFactory = new
22 SqlSessionFactoryBuilder().build(inputStream);
23         //3. 使用工厂生产SqlSession对象
24         SqlSession sqlSession=sqlSessionFactory.openSession();
25         //4. 使用SqlSession创建Dao接口的代理对象
26         UserMapper userMapper=sqlSession.getMapper(UserMapper.class);
27         //5. 使用代理对象执行方法
28         List<User> users=userMapper.selectAll();
29         System.out.println(users);
30         //6. 释放资源
31         sqlSession.close();
32         inputStream.close();
33     }
34 }
```

### 三、根据Id查询用户信息- `selectById`

1.重构 `UserMapper` 接口，加入 `selectById` 方法。

```
1 package com.zsh.mapper;
2
3 import com.zsh.domain.User;
4 import java.util.List;
5
6 public interface UserMapper {
7     List<User> selectAll();
8
9     User selectById(Integer id);
10 }
```

2.重构 `UserMapper.xml`，加入 `selectById` 节点

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.UserMapper">
6     <select id="selectAll" resultType="com.zsh.domain.User">
7         select * from user
8     </select>
9
10    <select id="selectById" resultType="com.zsh.domain.User">
11        select * from user where id=#{id}
12    </select>
13 </mapper>
```

3.重构 `UserTest.java`（使用了 `@Before`、`@After` 注解来优化测试类代码，大大减少了重复累赘的代码）

```
1 package com.zsh.test;
2
3 import com.zsh.domain.User;
```

```
4 import com.zsh.mapper.UserMapper;
5 import org.apache.ibatis.io.Resources;
6 import org.apache.ibatis.session.SqlSession;
7 import org.apache.ibatis.session.SqlSessionFactory;
8 import org.apache.ibatis.session.SqlSessionFactoryBuilder;
9 import org.junit.After;
10 import org.junit.Before;
11 import org.junit.Test;
12 import java.io.IOException;
13 import java.io.InputStream;
14 import java.util.List;
15
16 public class UserTest {
17     private InputStream inputStream;
18     private SqlSessionFactory sqlSessionFactory;
19     private SqlSession sqlSession;
20     private UserMapper userMapper;
21
22     @Before
23     public void init() throws IOException {
24         System.out.println("Before=====");
25         //1. 读取配置文件--在resources文件夹下可以直接读到
26         inputStream = Resources.getResourceAsStream("mybatis-
config.xml");
27         //2. 创建SqlSessionFactory工厂
28         sqlSessionFactory = new
SqlSessionFactoryBuilder().build(inputStream);
29         //3. 使用工厂生产SqlSession对象
30         sqlSession=sqlSessionFactory.openSession();
31         //4. 使用SqlSession创建Dao接口的代理对象
32         userMapper=sqlSession.getMapper(UserMapper.class);
33     }
34
35     @After
36     public void destroy() throws IOException {
37         System.out.println("After=====");
38         //释放资源
39         sqlSession.close();
40         inputStream.close();
41     }
42
43     @Test
44     public void testSelectAll() throws IOException {
45         //使用代理对象执行方法
46         List<User> users=userMapper.selectAll();
```

```

47         System.out.println(users);
48     }
49
50     @Test
51     public void testSelectById() throws IOException {
52         //使用代理对象执行方法
53         System.out.println(userMapper.selectById(46));
54     }
55 }

```

## 四、核心配置文件

MyBatis 的配置文件包含了会深深影响 MyBatis 行为的设置和属性信息。配置文档的顶层结构如下：

- configuration (配置)
  - properties (属性) ← 配置数据库连接文件
  - settings (设置)
  - typeAliases (类型别名) ← 配置XML映射文件返回类型别名
  - typeHandlers (类型处理器)
  - objectFactory (对象工厂)
  - plugins (插件)
  - environments (环境配置) ← 可配置多个开发环境或测试环境
    - environment (环境变量)
      - transactionManager (事务管理器)
      - dataSource (数据源)
  - databaseIdProvider (数据库厂商标识)
  - mappers (映射器) ← 配置XML映射文件

### 1-properties (属性)

- 创建 `jdbcConfig.properties` 文件

```

1 jdbc.driver=com.mysql.jdbc.Driver
2 jdbc.url=jdbc:mysql://localhost:3306/mybatis?characterEncoding=utf8
3 jdbc.username=root
4 jdbc.password=123456

```

- 重构主配置文件 `mybatis-config.xml` (注意连接数据库的4个基本信息要修改)

```
1 <configuration>
2     <!-- 配置属性 -->
3     <properties resource="jdbcConfig.properties"></properties>
4
5     <!-- 配置环境 -->
6     <environments default="mysql">
7         <!-- 配置mysql的环境-->
8         <environment id="mysql">
9             <!-- 配置事务的类型-->
10            <transactionManager type="JDBC"></transactionManager>
11            <!-- 配置数据源（连接池） -->
12            <dataSource type="POOLED">
13                <!-- 配置连接数据库的4个基本信息 -->
14                <property name="driver" value="${jdbc.driver}">
15                </property>
16                <property name="url" value="${jdbc.url}"></property>
17                <property name="username" value="${jdbc.username}">
18                </property>
19                <property name="password" value="${jdbc.password}">
20                </property>
21            </dataSource>
22        </environment>
23    </environments>
24    ...
25</configuration>
```

## 2-typeAliases (类型别名)

注意节点的位置顺序不能写错 (🔗`properties` (属性) -> 🔗`settings` (设置) -> 🔗`typeAliases` (类型别名) )

方式一：配置在主配置文件 `configuration` 节点之下

```

1 <!--使用typeAliases配置别名，它只能配置实体类的别名 -->
2 <typeAliases>
3 <!--
4     typeAlias用于配置别名，指定一个对象实体。type属性指定的是实体类全限定类名。
5     alias属性指定别名，当指定了别名就不再区分大小写
6 -->
7     <typeAlias type="com.zsh.domain.User" alias="User"></typeAlias>
8 </typeAliases>

```

## 方式二：扫包配置（推荐使用）

```

1 <typeAliases>
2     <package name="com.zsh.domain"/>
3 </typeAliases>

```

## 3-environments（环境配置）

在核心配置文件的 `environments` 标签中其实是可以配置多个 `environment`，使用 `id` 给每段环境起名，在 `environments` 中使用 `default='环境id'` 来指定使用哪个环境的配置。我们一般就配置一个 `environment` 即可

### 参考代码

```

1 <!--配置环境-->
2 <environments default="development">
3     <!--配置开发环境-->
4     <environment id="development">
5         <transactionManager type="JDBC"/>
6         <dataSource type="POOLED">
7             <property name="driver" value="${jdbc.driver}"/>
8             <property name="url" value="${jdbc.url}"/>
9             <property name="username" value="${jdbc.username}"/>
10            <property name="password" value="${jdbc.password}"/>
11        </dataSource>
12    </environment>
13
14    <!--配置测试环境-->
15    <environment id="test">
16        <transactionManager type="JDBC"/>

```

```
17         <dataSource type="POOLED">
18             <property name="driver" value="${jdbc.driver}"/>
19             <property name="url" value="${jdbc.url}"/>
20             <property name="username" value="${jdbc.username}"/>
21             <property name="password" value="${jdbc.password}"/>
22         </dataSource>
23     </environment>
24 </environments>
```

## 4-mappers（映射器）

配置 `<package>`，那么就会自动包含 `mapper` 包下的所有 `xml`，无需再一一配置映射器。

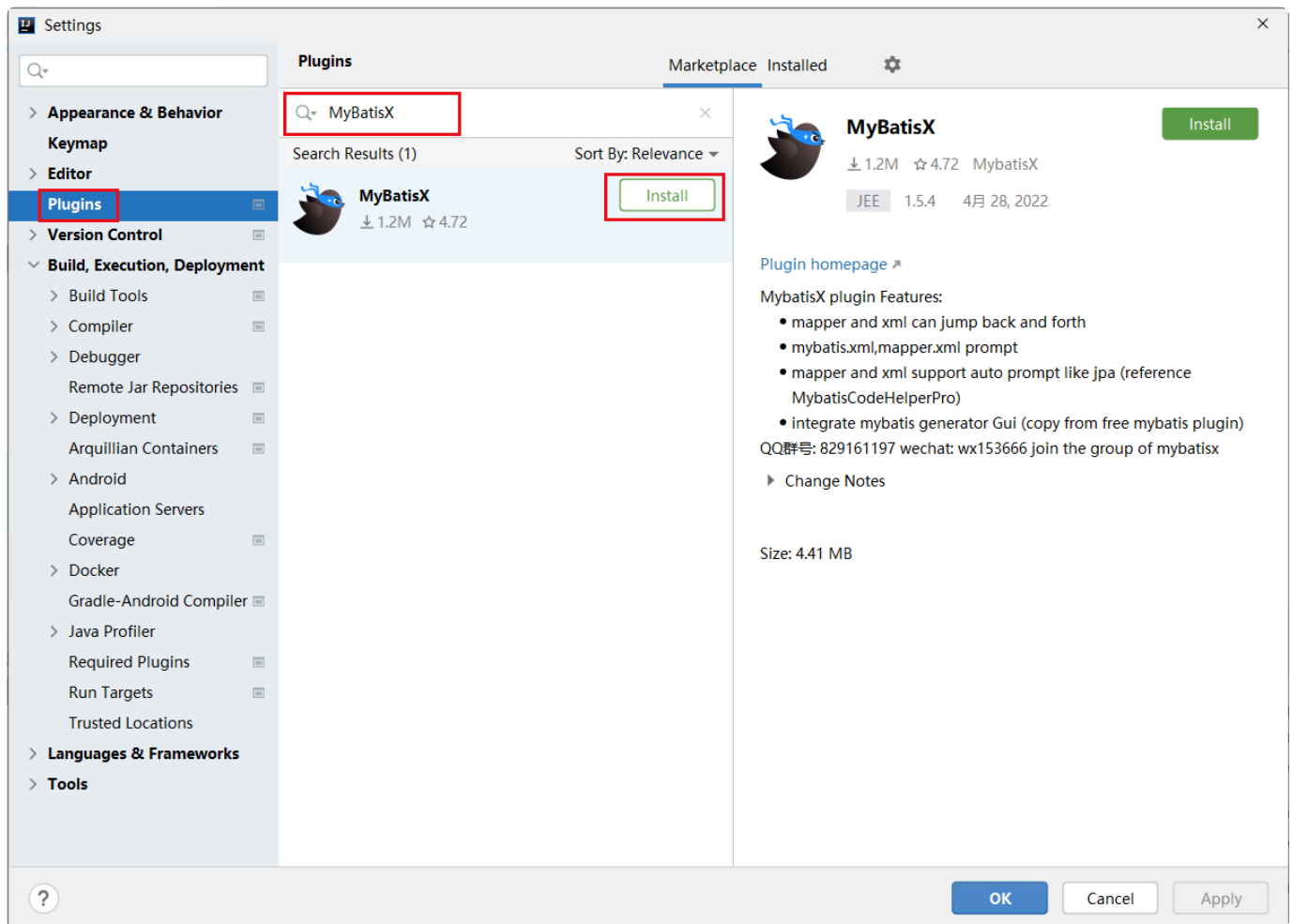
参考代码

```
1 <!-- 配置映射器 -->
2     <mappers>
3         <!--
4             <mapper resource="com/zsh/mapper/UserMapper.xml"/>
5         -->
6         <package name="com.zsh.mapper"/>
7     </mappers>
```

---

## 五、开发利器

### 1.MyBatisX



## 2.lombok

加入依赖坐标

```
1      <dependency>
2          <groupId>org.projectlombok</groupId>
3          <artifactId>lombok</artifactId>
4          <version>1.18.20</version>
5      </dependency>
```

实例类

```
1 package com.itbbj.domain;
2
3 import lombok.Data;
4
5 @Data//自动生成get&set方法
6 @NoArgsConstructor//自动生成无参构造器
```

```
7 @AllArgsConstructor//自动生成包含全部参数的有参构造器
8 public class User {
9     private Integer id;
10    private String username;
11    private String password;
12    private String gender;
13    private String address;
14 }
```

## 六、CRUD操作

### A.查询

#### A-1 resultMap 应用

##### A-1-1 案例需求：查询brand表中所有数据

|   | id | brand_name | company_name | ordered | description | status |
|---|----|------------|--------------|---------|-------------|--------|
| 1 | 1  | 格力         | 格力科技有限公司     | 1       | 格力改变世界      | 0      |
| 2 | 2  | 华为         | 华为技术有限公司     | 2       | 华为科技世界之巔    | 1      |
| 3 | 3  | 美的         | 美的科技有限公司     | 3       | 美的改变世界      | 1      |

##### A-1-2 实现步骤：

Step1.在实体层 `com.zsh.domain` 中创建实体类 `Brand` ,参考数据库中 `brand` 表的相应字段编码。

**注意事项：**留意属性名称和数据库字段是否一致

思考会造成什么现象？会造成查询出来的数据中，`brand_name` 和 `company_name` 均为 `null`

```

1  @Data
2  public class Brand {
3      private Integer id;
4      private String brandName;
5      private String companyName;
6      private Integer ordered;
7      private String description;
8      //1-启用状态 0-禁用状态
9      private Integer status;
10 }

```

Step2.在数据库映射层 `com.zsh.mapper` 中创建接口 `BrandMapper` ,编码。

```

1  package com.zsh.mapper;
2
3  import com.zsh.domain.Brand;
4  import java.util.List;
5
6  public interface BrandMapper {
7      List<Brand> selectAll();
8  }

```

Step3.在 `resources` 目录下创建映射配置文件 `BrandMapper.xml`

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <select id="selectAll" resultType="Brand">
7          select * from brand;
8      </select>
9  </mapper>

```

Step4.在 `test-java-com.zsh.test` 目录下创建 `BrandTest` 类，编码 `testSeleteAll()`。

```

1  package com.zsh.test;
2
3  import com.zsh.domain.Brand;
4  import com.zsh.mapper.BrandMapper;
5  import org.apache.ibatis.io.Resources;

```

```
6 import org.apache.ibatis.session.SqlSession;
7 import org.apache.ibatis.session.SqlSessionFactory;
8 import org.apache.ibatis.session.SqlSessionFactoryBuilder;
9 import org.junit.After;
10 import org.junit.Before;
11 import org.junit.Test;
12 import java.io.IOException;
13 import java.io.InputStream;
14 import java.util.List;
15
16 public class BrandTest {
17     private InputStream inputStream;
18     private SqlSessionFactory sqlSessionFactory;
19     private SqlSession sqlSession;
20     private BrandMapper brandMapper;
21
22     @Before
23     public void init() throws IOException {
24         System.out.println("Before=====");
25         //1.读取配置文件--在resources文件夹下可以直接读到
26         inputStream = Resources.getResourceAsStream("mybatis-
config.xml");
27         //2.创建SqlSessionFactory工厂
28         sqlSessionFactory = new
SqlSessionFactoryBuilder().build(inputStream);
29         //3.使用工厂生产SqlSession对象
30         sqlSession=sqlSessionFactory.openSession();
31         //4.使用SqlSession创建Dao接口的代理对象
32         brandMapper=sqlSession.getMapper(BrandMapper.class);
33     }
34
35     @After
36     public void destroy() throws IOException {
37         System.out.println("After=====");
38         //释放资源
39         sqlSession.close();
40         inputStream.close();
41     }
42
43     @Test
44     public void testSelectAll() throws IOException {
45         //使用代理对象执行方法
46         List<Brand> brands=brandMapper.selectAll();
47         for (Brand brand : brands) {
48             System.out.println(brand);
49         }
50     }
51 }
```

```

49     }
50 }
51 }

```

运行结果截图如下：

```

[DEBUG] 00:10:40.496 [main] c.z.m.B.selectAll - <==      Total: 3
Brand(id=1, brandName=null, companyName=null, ordered=1, description=格力改变世界, status=0)
Brand(id=2, brandName=null, companyName=null, ordered=2, description=华为科技世界之巅, status=1)
Brand(id=3, brandName=null, companyName=null, ordered=3, description=美的改变世界, status=1)

```

### A-1-3 对于 brand\_name、company\_name 显示为 null 的解决方案：

重构 BrandMapper.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <resultMap id="brandMap" type="Brand">
7          <id column="id" property="id"></id>
8          <result column="brand_name" property="brandName"></result>
9          <result column="company_name" property="companyName"></result>
10         <result column="ordered" property="ordered"></result>
11         <result column="description" property="description"></result>
12         <result column="status" property="status"></result>
13     </resultMap>
14     <select id="selectAll" resultMap="brandMap">
15         select * from brand;
16     </select>
17 </mapper>

```

可以简化，只写命名不同的属性

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <resultMap id="brandMap" type="Brand">
7          <id column="id" property="id"></id>

```

```

8         <result column="brand_name" property="brandName"></result>
9         <result column="company_name" property="companyName"></result>
10    </resultMap>
11    <select id="selectAll" resultMap="brandMap">
12        select * from brand;
13    </select>
14 </mapper>

```

运行结果截图如下：

```

[DEBUG] 23:50:43.855 [main] c.z.m.B.selectAll - ==> Parameters:
[DEBUG] 23:50:43.856 [main] c.z.m.B.selectAll - <==      Total: 3
Brand(id=1, brandName=格力, companyName=格力科技有限公司, ordered=1, description=格力改变世界, status=0)
Brand(id=2, brandName=华为, companyName=华为技术有限公司, ordered=2, description=华为科技世界之巅, status=1)
Brand(id=3, brandName=美的, companyName=美的科技有限公司, ordered=3, description=美的改变世界, status=1)
...

```

## A-2 模糊查询

**A-2-1 案例需求：**根据公司名称进行全匹配模糊查询，查询公司名称中包含“科技有限”的数据信息

| WHERE |    |            | ORDER BY     |         |             |        |
|-------|----|------------|--------------|---------|-------------|--------|
|       | id | brand_name | company_name | ordered | description | status |
| 1     | 1  | 格力         | 格力科技有限公司     | 1       | 格力改变世界      | 0      |
| 2     | 2  | 华为         | 华为技术有限公司     | 2       | 华为科技世界之巅    | 1      |
| 3     | 3  | 美的         | 美的科技有限公司     | 3       | 美的改变世界      | 1      |

### A-2-2 实现方式

方式一（不推荐）：`brandMapper.selectByCompanyName("%科技有限%")`

步骤1：在 `BrandMapper` 层创建 `selectByCompanyName` 方法

```

1 package com.zsh.mapper;
2 import com.zsh.domain.Brand;
3 import java.util.List;
4
5 public interface BrandMapper {
6     List<Brand> selectAll();
7
8     List<Brand> selectByCompanyName(String companyName);
9 }

```

步骤2：重构 `BrandMapper.xml` // # 占位符

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.BrandMapper">
6     <resultMap id="brandMap" type="Brand">
7         <id column="id" property="id"></id>
8         <result column="brand_name" property="brandName"></result>
9         <result column="company_name" property="companyName"></result>
10        <result column="ordered" property="ordered"></result>
11        <result column="description" property="description"></result>
12        <result column="status" property="status"></result>
13    </resultMap>
14
15    <select id="selectByCompanyName" resultMap="brandMap">
16        select * from brand where company_name like #{companyName};
17    </select>
18 </mapper>

```

步骤3：创建测试方法 `testSeleteByCompanyName` 进行测试

```

1 @Test
2 public void testSelectByCompanyName() throws IOException{
3     List<Brand> brands=brandMapper.selectByCompanyName("%科技有限%");
4     for (Brand brand : brands) {
5         System.out.println(brand);
6     }
7 }

```

运行结果截图如下：

```
[DEBUG] 00:33:41.303 [main] c.z.m.B.selectByCompanyName - ==> Preparing: select * from brand where company_name like ?;
[DEBUG] 00:33:41.328 [main] c.z.m.B.selectByCompanyName - ==> Parameters: %科技有限%(String)
[DEBUG] 00:33:41.355 [main] c.z.m.B.selectByCompanyName - <==          Total: 2
Brand(id=1, brandName=格力, companyName=格力科技有限公司, ordered=1, description=格力改变世界, status=0)
Brand(id=3, brandName=美的, companyName=美的科技有限公司, ordered=3, description=美的改变世界, status=1)
```

方式二（不推荐）：`like '%${companyName}%'`

步骤1：在 `BrandMapper` 层创建 `selectByCompanyName` 方法：同方式一步骤1

步骤2：重构 `BrandMapper.xml` //字符串拼接

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.BrandMapper">
6     <resultMap id="brandMap" type="Brand">
7         <id column="id" property="id"></id>
8         <result column="brand_name" property="brandName"></result>
9         <result column="company_name" property="companyName"></result>
10        <result column="ordered" property="ordered"></result>
11        <result column="description" property="description"></result>
12        <result column="status" property="status"></result>
13    </resultMap>
14
15    <select id="selectByCompanyName" resultMap="brandMap">
16        select * from brand where company_name like '%${companyName}%';
17    </select>
18 </mapper>
```

步骤3：创建测试方法 `testSeleteByCompanyName` 进行测试

```

1      @Test
2      public void testSelectByCompanyName() throws IOException{
3          List<Brand> brands=brandMapper.selectByCompanyName("科技有限");//
        无需%号
4          for (Brand brand : brands) {
5              System.out.println(brand);
6          }
7      }

```

运行结果截图如下：

```

[DEBUG] 00:42:18.630 [main] c.z.m.B.selectByCompanyName - ==> Preparing: select * from brand where company_name like '%科技有限
[DEBUG] 00:42:18.655 [main] c.z.m.B.selectByCompanyName - ==> Parameters:
[DEBUG] 00:42:18.687 [main] c.z.m.B.selectByCompanyName - <==          Total: 2
Brand(id=1, brandName=格力, companyName=格力科技有限公司, ordered=1, description=格力改变世界, status=0)
Brand(id=3, brandName=美的, companyName=美的科技有限公司, ordered=3, description=美的改变世界, status=1)

```

方式三（推荐）：`concat('%',#{companyName},'%')`

步骤1：在 `BrandMapper` 层创建 `selectByCompanyName` 方法：同方式一步骤1

步骤2：重构 `BrandMapper.xml`

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <resultMap id="brandMap" type="Brand">
7          <id column="id" property="id"></id>
8          <result column="brand_name" property="brandName"></result>
9          <result column="company_name" property="companyName"></result>
10         <result column="ordered" property="ordered"></result>
11         <result column="description" property="description"></result>
12         <result column="status" property="status"></result>
13     </resultMap>
14
15     <select id="selectByCompanyName" resultMap="brandMap">
16         select * from brand where company_name like concat('%',#
17         {companyName},'%');
18     </select>

```

步骤3：创建测试方法 `testSeleteByCompanyName` 进行测试：同方式二步骤3

运行结果截图如下：

```
[DEBUG] 00:45:54.345 [main] c.z.m.B.selectByCompanyName - ==> Preparing: select * from brand where company_name like concat('%',?, '%');
[DEBUG] 00:45:54.369 [main] c.z.m.B.selectByCompanyName - ==> Parameters: 科技有限(String)
[DEBUG] 00:45:54.396 [main] c.z.m.B.selectByCompanyName - <==          Total: 2
Brand(id=1, brandName=格力, companyName=格力科技有限公司, ordered=1, description=格力改变世界, status=0)
Brand(id=3, brandName=美的, companyName=美的科技有限公司, ordered=3, description=美的改变世界, status=1)
```

## A-2-3 总结

- 1 方式一：封装数据时耦合业务需求（不推荐）
- 2 `brandMapper.selectByCompanyName("%科技有限%");`
- 3 方式二：like `'%${companyName}%'`；（不推荐）
- 4 `'%${companyName}%'`表示字符串连接，相当于`'%'+XXX+'%'`，但是会产生SQL注入，有安全漏洞
- 5 方式三：like `concat('%',#{companyName}, '%')` - 推荐

## A-3 多参数查询

**A-3-1 案例需求：**根据公司名称及品牌状态进行全匹配模糊查询，查询公司名称中包含“科技有限”及状态为‘0’的数据信息

|   | id | brand_name | company_name | ordered | description | status |
|---|----|------------|--------------|---------|-------------|--------|
| 1 | 1  | 格力         | 格力科技有限公司     | 1       | 格力改变世界      | 0      |
| 2 | 2  | 华为         | 华为技术有限公司     | 2       | 华为科技世界之巅    | 1      |
| 3 | 3  | 美的         | 美的科技有限公司     | 3       | 美的改变世界      | 1      |

## A-3-2 实现方法

方法一：使用 `@param` 注解

步骤1：在BrandMapper层创建selectByCompanyNameAndStatus方法

```
1 package com.zsh.mapper;
2
3 import com.zsh.domain.Brand;
4 import org.apache.ibatis.annotations.Param;
5 import java.util.List;
6
7 public interface BrandMapper {
8     List<Brand> selectAll();
9     List<Brand> selectByCompanyName(String companyName);
10
11     List<Brand> selectByCompanyNameAndStatus
12         (@Param("companyName") String companyName,
13          @Param("status") Integer status);
14 }
```

步骤2：重构BrandMapper.xml

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.BrandMapper">
6     <resultMap id="brandMap" type="Brand">
7         <id column="id" property="id"></id>
8         <result column="brand_name" property="brandName"></result>
9         <result column="company_name" property="companyName"></result>
10        <result column="ordered" property="ordered"></result>
11        <result column="description" property="description"></result>
12        <result column="status" property="status"></result>
13    </resultMap>
14
15    <select id="selectByCompanyNameAndStatus" resultMap="brandMap">
16        select *
17        from brand where company_name like concat('%',#
18        {companyName}, '%')
19        and status=#{status}
20    </select>
```

### 步骤3：创建测试方法testSelectByCompanyNameAndStatus进行测试

```

1      @Test
2      public void testSelectByCompanyNameAndStatus() throws IOException {
3          List<Brand> brands=brandMapper.selectByCompanyNameAndStatus("科
科技有限",0);
4          System.out.println(brands);
5      }

```

### 运行结果截图如下：

```

[DEBUG] 00:55:51.262 [main] c.z.m.B.selectByCompanyNameAndStatus - ==> Preparing: select * from brand where com
[DEBUG] 00:55:51.293 [main] c.z.m.B.selectByCompanyNameAndStatus - ==> Parameters: 科技有限(String), 0(Integer)
[DEBUG] 00:55:51.319 [main] c.z.m.B.selectByCompanyNameAndStatus - <==      Total: 1
[Brand(id=1, brandName=格力, companyName=格力科技有限公司, ordered=1, description=格力改变世界, status=0)]

```

### 方法二：通过封装实体对象数据进行传参

#### 步骤1：在BrandMapper层创建selectByBrand方法

```

1  package com.zsh.mapper;
2
3  import com.zsh.domain.Brand;
4  import org.apache.ibatis.annotations.Param;
5  import java.util.List;
6
7  public interface BrandMapper {
8      List<Brand> selectAll();
9      List<Brand> selectByCompanyName(String companyName);
10     List<Brand> selectByCompanyNameAndStatus
11         (@Param("companyName") String companyName,
12          @Param("status") Integer status);
13
14     List<Brand> selectByBrand(Brand brand);
15 }

```

#### 步骤2：重构BrandMapper.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <resultMap id="brandMap" type="Brand">
7          <id column="id" property="id"></id>
8          <result column="brand_name" property="brandName"></result>
9          <result column="company_name" property="companyName"></result>
10         <result column="ordered" property="ordered"></result>
11         <result column="description" property="description"></result>
12         <result column="status" property="status"></result>
13     </resultMap>
14
15     <select id="selectByBrand" resultMap="brandMap">
16         select *
17         from brand where company_name like concat('%',#
18 {companyName}, '%')
19         and status=#{status}
20     </select>
21 </mapper>

```

### 步骤3：创建测试方法testSeleteByBrand进行测试

```

1  @Test
2  public void testSelectByBrand() throws IOException {
3      Brand brand=new Brand();
4      brand.setCompanyName("科技有限");
5      brand.setStatus(0);
6      List<Brand> brands=brandMapper.selectByBrand(brand);
7      System.out.println(brands);
8  }

```

### 运行结果截图如下：

```

[DEBUG] 01:05:41.664 [main] c.z.m.B.selectByBrand - ==> Preparing: select * from brand where company_name like concat('%',?, '%') and status=?
[DEBUG] 01:05:41.687 [main] c.z.m.B.selectByBrand - ==> Parameters: 科技有限(String), 0(Integer)
[DEBUG] 01:05:41.716 [main] c.z.m.B.selectByBrand - <==      Total: 1
[Brand(id=1, brandName=格力, companyName=格力科技有限公司, ordered=1, description=格力改变世界, status=0)]

```

### 方法三：通过封装 Map 对象数据进行传参

## 步骤1：在BrandMapper接口层创建selectByCondition方法

```
1 package com.zsh.mapper;
2
3 import com.zsh.domain.Brand;
4 import org.apache.ibatis.annotations.Param;
5 import java.util.List;
6 import java.util.Map;
7
8 public interface BrandMapper {
9     List<Brand> selectAll();
10    List<Brand> selectByCompanyName(String companyName);
11    List<Brand> selectByCompanyNameAndStatus
12        (@Param("companyName") String companyName,
13         @Param("status") Integer status);
14    List<Brand> selectByBrand(Brand brand);
15
16    List<Brand> selectByCondition(Map map);
17 }
```

## 步骤2：重构BrandMapper.xml

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.BrandMapper">
6     <resultMap id="brandMap" type="Brand">
7         <id column="id" property="id"></id>
8         <result column="brand_name" property="brandName"></result>
9         <result column="company_name" property="companyName"></result>
10        <result column="ordered" property="ordered"></result>
11        <result column="description" property="description"></result>
12        <result column="status" property="status"></result>
13    </resultMap>
14
15    <select id="selectByCondition" resultMap="brandMap">
16        select *
17        from brand where company_name like concat('%',#
18        {companyName}, '%')
19        and status=#{status}
20    </select>
```

### 步骤3：创建测试方法testSeleteByCondition进行测试

```

1      @Test
2      public void testSelectByCondition() throws IOException {
3          Map map=new HashMap();
4          map.put("companyName","科技有限");
5          map.put("status",0);
6          List<Brand> brands=brandMapper.selectByCondition(map);
7          System.out.println(brands);
8      }

```

### 运行结果截图如下：

```

[DEBUG] 01:13:07.421 [main] c.z.m.B.selectByCondition - ==> Preparing: select * from brand where company_name like concat('%',?, '%') and status=?
[DEBUG] 01:13:07.450 [main] c.z.m.B.selectByCondition - ==> Parameters: 科技有限(String), 0(Integer)
[DEBUG] 01:13:07.486 [main] c.z.m.B.selectByCondition - <==          Total: 1
[Brand(id=1, brandName=格力, companynName=格力科技有限公司, ordered=1, description=格力改变世界, status=0)]

```

### A-3-3 总结：

```

1  方式1-直接通过参数-缺点：参数一变，就要修改接口方法
2  selectByCompanyNameAndStatus(@Param("companynName") String companyName,
3  ...)
4  方式2：通过封装实体对象数据进行传参数-缺点：如果数据来自多个对象，也要修改接口方法
5  selectByBrand(Brand brand)
6  方式3：通过封装Map对象数据进行传参数-推荐
7  selectByCondition(Map map)

```

# A-4 动态查询-where-if

## A-4-1 案例需求

需求：高级查询中，条件是动态的，有可能需要查询，有可能不需要查询

商品管理

订单管理

物流管理

促销管理

文章管理

权限管理

VIP原型

下载原型

亮

首页

当前状态

应用

企业名称

企业名称

品牌名称

华为

搜索

数据列表

删除

实际开发中，企业名称有可能不查询

| <input type="checkbox"/> | 序号  | 品牌LOGO                                                                            | 品牌名称 | 企业名称    | 排序 | 当前状态                                | 操作         |
|--------------------------|-----|-----------------------------------------------------------------------------------|------|---------|----|-------------------------------------|------------|
| <input type="checkbox"/> | 010 |  | 三只松鼠 | 这里是企业名称 | 18 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| <input type="checkbox"/> | 009 |  | 优衣库  | 这里是企业名称 | 14 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| <input type="checkbox"/> | 008 |  | 小米   | 这里是企业名称 | 12 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| <input type="checkbox"/> | 007 |  | 阿迪达斯 | 这里是企业名称 | 12 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| <input type="checkbox"/> | 006 |  | 百草味  | 这里是企业名称 | 3  | <input type="checkbox"/>            | 删除 编辑 查看详情 |

## A-4-2 案例代码

步骤1：重构映射文件xml-`<select id="selectByCondition">...<select>`

```
1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <resultMap id="brandMap" type="Brand">
7          <id column="id" property="id"></id>
8          <result column="brand_name" property="brandName"></result>
9          <result column="company_name" property="companyName"></result>
10         <result column="ordered" property="ordered"></result>
11         <result column="description" property="description"></result>
12         <result column="status" property="status"></result>
13     </resultMap>
14
15     <select id="selectByCondition1" resultMap="brandMap">
16         select * from brand
17         <where>
18             <if test="companyName!=null and companyName!=''">
19                 and company_name like concat('%',{companyName},'%')
```

```
20         </if>
21         <if test="brandName!=null and brandName!=''">
22             and brand_name like concat('%',{brandName},'%')
23         </if>
24         <if test="status!=null">
25             and status=#{status}
26         </if>
27     </where>
28 </select>
29 </mapper>
```

步骤2：测试代码（略）

---

### A-4-3 总结

- 1 if 标签：条件判断
- 2 - test 属性：逻辑表达式 与是and,或是or
- 3
- 4 where 标签
- 5 - 作用：
- 6 - 替换where关键字
- 7 - 会动态的去掉第一个条件前的 and（如果有and的话，没有也不会报错）
- 8 - 如果所有的参数没有值则不加where关键字

---

## A-5 动态查询-choose-when

### A-5-1 案例需求

## 需求：高级查询中，查询条件是单条的，可以动态选择一个查询条件

请选择条件: 当前状态, 企业名称, 品牌名称

单条件动态查询

| 序号  | 品牌LOGO | 品牌名称  | 企业名称    | 排序 | 请选择条件                               | 操作         |
|-----|--------|-------|---------|----|-------------------------------------|------------|
| 010 |        | 三只松鼠  | 这里是企业名称 | 19 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| 009 |        | 优衣库   | 这里是企业名称 | 17 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| 008 |        | 小米    | 这里是企业名称 | 9  | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| 007 |        | 阿迪达斯  | 这里是企业名称 | 11 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| 006 |        | 百草味   | 这里是企业名称 | 8  | <input type="checkbox"/>            | 删除 编辑 查看详情 |
| 005 |        | zara  | 这里是企业名称 | 14 | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |
| 004 |        | 美特斯邦威 | 这里是企业名称 | 1  | <input checked="" type="checkbox"/> | 删除 编辑 查看详情 |

实现方法：使用 **choose (when, otherwise)** 标签和java的 **switch (case, default)** 一样一样的

### A-5-2 案例代码

步骤1：重构映射文件xml- `<select id="selectByCondition">...</select>`

若一个when满足了，则后续的所有when都会忽略

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.BrandMapper">
6     <resultMap id="brandMap" type="Brand">
7         <id column="id" property="id"></id>
8         <result column="brand_name" property="brandName"></result>
9         <result column="company_name" property="companyName"></result>
10        <result column="ordered" property="ordered"></result>
11        <result column="description" property="description"></result>
12        <result column="status" property="status"></result>
13    </resultMap>
14
15    <select id="selectByCondition1" resultMap="brandMap">
16        select * from brand
17        <where>
18            <choose>
19                <when test="companyName!=null and companyName!=''">
```

```
20         and company_name like concat('%',#{
{companyName},'%')
21     </when>
22     <when test="brandName!=null and brandName!=''">
        and brand_name like concat('%',#{brandName},'%')
23     </when>
24     <when test="status!=null">
        and status=#{status}
25     </when>
26     </choose>
27 </where>
28 </select>
29 </mapper>
```

步骤2：创建测试代码（略）

## A-6 SQL语句特殊符号处理

### A-6-1 案例需求：查询id < 45的用户信息

### A-6-2 总结

#### 总结：方式一：使用转义字符

| 符号   | 原始字符 | 转义字符   |
|------|------|--------|
| 大于   | >    | &gt;   |
| 大于等于 | >=   | &gt;=  |
| 小于   | <    | &lt;   |
| 小于等于 | <=   | &lt;=  |
| 和    | &    | &amp;  |
| 单引号  | '    | &apos; |
| 双引号  | "    | &quot; |

#### 方式二：使用CD <![CDATA[内容]]>

## B.新增

### B-1 单行新增

步骤1-在 `BrandMapper` 层创建insert方法

```
1 package com.zsh.mapper;
2
3 import com.zsh.domain.Brand;
4
5 public interface BrandMapper {
6     int insert(Brand brand);
7 }
```

步骤2-重构 `BrandMapper.xml`

参考代码-普通新增

```
1 <insert id="insert">
2     insert into brand (brand_name, company_name, ordered,
3     description, status)
4     values (#{brandName},#{companyName},#{ordered},#{description},#
5     {status});
6 </insert>
```

参考代码-返回新增成功对象的id

```
1 <insert id="insert" useGeneratedKeys="true" keyProperty="id">
2     insert into brand (brand_name, company_name, ordered,
3     description, status)
4     values (#{brandName},#{companyName},#{ordered},#{description},#
5     {status});
6 </insert>
```

步骤3-创建 `testInsert` 测试方法进行测试

```
1      @Test
2      public void testInsert() throws IOException {
3          Brand brand = new Brand();
4          brand.setBrandName("小米");
5          brand.setCompanyName("小米科技有限公司");
6          brand.setOrdered(4);
7          brand.setDescription("小米科技天下无敌");
8          brand.setStatus(1);
9          brandMapper.insert(brand);
10     }
```

---

## B-2 批量新增

步骤1-在 `BrandMapper` 层创建 `insertBatch` 方法

```
1  package com.zsh.mapper;
2
3  import com.zsh.domain.Brand;
4  import java.util.List;
5
6  public interface BrandMapper {
7      int insertBatch(@Param("brands") List<Brand> brands);
8  }
```

步骤2-重构 `BrandMapper.xml`

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <insert id="insertBatch">
7          insert into brand values
8          <foreach collections="brand" item="brand" separator=",">
9              ({brand.brandName},{brand.companyName},{brand.ordered},{#
10 {brand.decription},{brand.status})
11          </foreach>
12      </insert>
13  </mapper>

```

### 步骤3-创建 `testInsertBatch` 测试方法进行测试

```

1  @Test
2  public void testInsertBatch() throws IOException {
3      Brand brand1 = new Brand();
4      brand1.setBrandName("小米");
5      brand1.setCompanyName("小米科技有限公司");
6      brand1.setOrdered(7);
7      brand1.setDescription("小米科技天下无敌");
8      brand1.setStatus(1);
9
10     Brand brand2 = new Brand();
11     brand2.setBrandName("联想");
12     brand2.setCompanyName("联想科技有限公司");
13     brand2.setOrdered(8);
14     brand2.setDescription("联想电脑牛逼");
15     brand2.setStatus(0);
16
17     brands.add(brand1);
18     brands.add(brand2);
19     brandMapper.insertBatch(brands);
20 }

```

## C.修改

### C-1 案例需求：根据品牌Id，修改品牌信息

---

### C-2 实现步骤

步骤1-在 `BrandMapper` 层创建 `updateById` 方法

```
1 package com.zsh.mapper;
2
3 import com.zsh.domain.Brand;
4
5 public interface BrandMapper {
6     void updateById(Brand brand);
7 }
```

步骤2-重构 `BrandMapper.xml` (动态修改)

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.BrandMapper">
6     <update id="updateById">
7         update brand
8         <set>
9             <if test="brandName!=null and brandName!=''">
10                 brand_name=#{brandName}
11             </if>
12             <if test="companyName!=null and companyName!=''">
13                 company_name=#{companyName}
14             </if>
15             <if test="ordered!=null">
16                 ordered=#{ordered}
17             </if>
18             <if test="description!=null and description!=''">
19                 description=#{description}
20             </if>
21             <if test="status!=null">
22                 status=#{status}
23             </if>
```

```
24         </set>
25         where id=#{id};
26     </update>
27 </mapper>
```

步骤3-创建 `testUpdateById` 测试方法

```
1     @Test
2     public void testUpdateById() throws IOException {
3         Brand brand=new Brand();
4         brand.setId(3);
5         brand.setBrandName("小米米");
6         brand.setCompanyName("小米米科技有限公司");
7         brand.setStatus(0);
8         brandMapper.updateById(brand);
9     }
```

## D.删除

### D-1 单行删除

步骤1-在 `BrandMapper` 层创建 `deleteById` 方法

```
1 package com.zsh.mapper;
2
3 public interface BrandMapper {
4     void deleteById(Integer id);
5 }
```

步骤2-重构 `BrandMapper.xml`

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <mapper namespace="com.zsh.mapper.BrandMapper">
6     <delete id="deleteById">
7         delete from brand where id=#{id};
8     </delete>
9 </mapper>

```

### 步骤3-创建 `testDeleteById` 测试方法

```

1 @Test
2 public void testDeleteById() throws IOException {
3     brandMapper.deleteById(8);
4 }

```

## D-2 批量删除

### 步骤1-在 `BrandMapper` 层创建 `deleteByIds` 方法

```

1 package com.zsh.mapper;
2
3 public interface BrandMapper {
4     void deleteByIds(Integer[] ids);
5 }

```

### 步骤2-重构BrandMapper.xml

#### 参考代码-方式1

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

```

```

5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <delete id="deleteByIds">
7          delete from brand
8          where id in(
9              <foreach collection="array" item="id" separator=",">
10                 #{id}
11             </foreach>
12         );
13     </delete>
14 </mapper>

```

## 参考代码-方式2

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.zsh.mapper.BrandMapper">
6      <delete id="deleteByIds">
7          delete from brand
8          where id in
9              <foreach collection="array" item="id" separator="," open="("
10             close=")">
11                 #{id}
12             </foreach>
13     </delete>
14 </mapper>

```

## 步骤3-创建 testDeleteByIds 测试方法

```

1  @Test
2  public void testDeleteByIds() throws IOException {
3      Integer[] ids={6,7};
4      brandMapper.deleteById(ids);
5  }

```

## E.注解实现CRUD（适合简单逻辑）

### E-1 需求-实现角色role表的增删改查

---

#### E-2 实现步骤

##### 步骤1-新增实体类 Role

```
1 package com.zsh.domain;
2
3 import lombok.Data;
4
5 @Data
6 public class Role {
7     private Integer id;
8     private String rolename;
9     private String roledesc;
10 }
11
```

##### 步骤2- RoleMapper

```
1 package com.zsh.mapper;
2
3 import com.zsh.domain.Role;
4 import org.apache.ibatis.annotations.Insert;
5 import org.apache.ibatis.annotations.Options;
6 import org.apache.ibatis.annotations.Select;
7 import java.util.List;
8
9 public interface RoleMapper {
10     @Select("select * from role")
11     List<Role> selectAll();
12
13     @Insert("insert into role values (#{id},#{rolename},#{roledesc});")
14     @Options(useGeneratedKeys = true, keyProperty = "id")
15     int insert(Role role);
16 }
```

##### 步骤3- RoleTest 测试方法（略，记得开启事务--

```
sqlSession=sqlSessionFactory.openSession(true) )
```

